

01. EDA & Log Preprocessing

본 노트북은 LogSentinel 프로젝트의 1~2단계에 해당하는 로그 데이터 탐색(EDA) 및 정규식 기반 텍스트 정제(Parsing), TF-IDF 벡터화 과정의 작동성을 검증하는 공간입니다.

```
In [1]: import os
import sys
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

# 프로젝트 루트 경로를 sys.path에 추가하여 src 패키지를 찾을 수 있도록 함
sys.path.append(os.path.abspath(os.path.join("..")))

# 작성한 모듈 임포트
from src.parser import LogParser

# 한글 폰트 설정 (추가됨)
import platform

if platform.system() == "Darwin": # Mac
    plt.rcParams["font.family"] = "AppleGothic"
elif platform.system() == "Windows": # Windows
    plt.rcParams["font.family"] = "Malgun Gothic"
plt.rcParams["axes.unicode_minus"] = False
```

1. 데이터 로드 및 정상/장애 통계 확인

BGL 로그의 1열이 - 이면 정상, 그 외의 태그이면 이상(Anomaly)을 의미합니다.

```
In [2]: # 경로 설정 (T7 외장 SSD 우선, 없으면 로컬 fallback)
SSD_PATH = "/Volumes/T7/LogSentinel_Data/raw/"
LOCAL_PATH = "./data/raw/"

def get_data_path(filename):
    ssd_file = os.path.join(SSD_PATH, filename)
    local_file = os.path.join(LOCAL_PATH, filename)
    if os.path.exists(ssd_file):
        return ssd_file
    return local_file

log_file_path = get_data_path("BGL_2k.log")
print(f"Using log file: {log_file_path}")
```

Using log file: /Volumes/T7/LogSentinel_Data/raw/BGL_2k.log

```
In [3]: # 로그 데이터 파싱하여 리스트업
data = []
with open(log_file_path, "r", encoding="utf-8") as f:
```

```

for line in f:
    parts = line.strip().split()
    if len(parts) < 10:
        continue
    label = parts[0]
    is_anomaly = 1 if label != "-" else 0
    message = " ".join(parts[9:])
    data.append({"label_raw": label, "is_anomaly": is_anomaly, "raw_message": message})

df = pd.DataFrame(data)
print(f"Total parsed logs: {len(df)}")
print(df["is_anomaly"].value_counts(normalize=True))

```

```

Total parsed logs: 2000
is_anomaly
0    0.9285
1    0.0715
Name: proportion, dtype: float64

```

In [4]: df

Out[4]:

	label_raw	is_anomaly	raw_message
0	-	0	instruction cache parity error corrected
1	-	0	instruction cache parity error corrected
2	-	0	instruction cache parity error corrected
3	-	0	instruction cache parity error corrected
4	-	0	63543 double-hummer alignment exceptions
...	...	...	...
1995	-	0	instruction cache parity error corrected
1996	-	0	instruction cache parity error corrected
1997	-	0	instruction cache parity error corrected
1998	-	0	instruction cache parity error corrected
1999	-	0	ciod: generated 128 core files for program /g/...

2000 rows x 3 columns

2. 정규식 파서 전처리 테스트

```

In [5]: parser = LogParser(max_features=500)
sample_log1 = df.iloc[1]["raw_message"]
sample_log2 = df.iloc[2]["raw_message"]
sample_log9 = df.iloc[9]["raw_message"]

print(f'sample_log1: {sample_log1}')
print(f'sample_log2: {sample_log2}')
print(f'sample_log9: {sample_log9}')

```

```
sample_log1: instruction cache parity error corrected
sample_log2: instruction cache parity error corrected
sample_log9: ciod: failed to read message prefix on control stream (CioStream socket to 172.16.96.116:33370)
```

```
In [6]: # 샘플 클리닝 테스트
sample_log = df.iloc[9]["raw_message"]
print("Raw:", sample_log)
print("Cleaned:", parser.clean_log(sample_log))
```

```
Raw: ciod: failed to read message prefix on control stream (CioStream socket to 172.16.96.116:33370)
Cleaned: ciod: failed to read message prefix on control stream (ciostream socket to [ip] : [num])
```

```
In [7]: # 전체 데이터 정제 적용
df["cleaned_message"] = df["raw_message"].apply(parser.clean_log)
print(df[["raw_message", "cleaned_message"]].head())
```

```
raw_message \
0 instruction cache parity error corrected
1 instruction cache parity error corrected
2 instruction cache parity error corrected
3 instruction cache parity error corrected
4 63543 double-hummer alignment exceptions

cleaned_message
0 instruction cache parity error corrected
1 instruction cache parity error corrected
2 instruction cache parity error corrected
3 instruction cache parity error corrected
4 [num] double-hummer alignment exceptions
```

3. TF-IDF 벡터화 실행

```
In [8]: tfidf_matrix = parser.fit_transform(df["cleaned_message"])
print("TF-IDF Matrix Shape:", tfidf_matrix.shape)
```

```
TF-IDF Matrix Shape: (2000, 500)
```

```
In [9]: tfidf_matrix
```

```
Out[9]: array([[0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               ...,
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.],
               [0., 0., 0., ..., 0., 0., 0.]], shape=(2000, 500))
```

```
In [10]: # 벡터라이저 저장
os.makedirs("/Volumes/T7/LogSentinel_Data/models", exist_ok=True)
parser.save_vectorizer("/Volumes/T7/LogSentinel_Data/models/vectorizer.pkl")
print("Vectorizer saved successfully.")
```

```
Vectorizer saved successfully.
```

